

图论

这是算法竞赛中最形象的内容，你敢和它对视三秒吗？

Rosaya

Oasis

目录

1. 基本概念	2
2. 图基础	7
2.1 图的遍历	8
2.2 二分图	9
2.3 有向无环图	12
2.4 习题	15
3. 最短路	16
3.1 可视化	17
3.2 Dijkstra	18
3.3 Hack Dijkstra	21
3.4 Bellman-Ford	23
3.5 Floyd	28
3.6 Johnson	29
3.7 习题	30

目录

1. 基本概念	2
2. 图基础	7
2.1 图的遍历	8
2.2 二分图	9
2.3 有向无环图	12
2.4 习题	15
3. 最短路	16
3.1 可视化	17
3.2 Dijkstra	18
3.3 Hack Dijkstra	21
3.4 Bellman-Ford	23
3.5 Floyd	28
3.6 Johnson	29
3.7 习题	30

- 图

一个二元组 $G = (V(G), E(G))$ (无歧义时简写为 $G = (V, E)$), 其中 V 是点集, $v \in V$ 称为点, E 是边集, $e \in E$ 称为边。一般记 $n = |V|$ 为图的点数, $m = |E|$ 为图的边数。当 V, E 均为有限集合时称为有限图, 否则称为无限图。

- 图

一个二元组 $G = (V(G), E(G))$ (无歧义时简写为 $G = (V, E)$), 其中 V 是点集, $v \in V$ 称为点, E 是边集, $e \in E$ 称为边。一般记 $n = |V|$ 为图的点数, $m = |E|$ 为图的边数。当 V, E 均为有限集合时称为有限图, 否则称为无限图。

- 边

图分为无向图, 有向图和混合图三种。对于无向图, 仅包含无向边, 记为 $e = (u, v)$, 此时 $(u, v) = (v, u)$, 表示 u, v 之间有边相连。对于有向图, 仅包含有向边, 记为 $e = (u, v)$, 表示有一条 u 指向 v 的边, 也可以记为 $u \rightarrow v$ 。对于混合图, 以上两类边都可以存在。记 u, v 为 e 的端点, 与 e 关联, u, v 在 G 中相邻。在有向图中 u 是 e 的起点, v 是 e 的终点。边可以有边权, 形式变为 $e = (u, v, w)$ 。

- 简单图

自环，即 $e = (u, u)$ ，**重边**，即 $e_1 = e_2 = (u, v)$ 。需要注意在无向图中 $e_1 = (u, v)$ ， $e_2 = (v, u)$ 也算重边而在有向图中不算。**简单图**指没有重边和自环的图，反之被称为**多重图**，若对任意 $u \neq v$ ， $(u, v) \in E$ ，则简单图 G 称为**完全图**。

需要注意，如果题目中没有注明是简单图，要考虑重边和自环带来的影响！

- 简单图

自环，即 $e = (u, u)$ ，**重边**，即 $e_1 = e_2 = (u, v)$ 。需要注意在无向图中 $e_1 = (u, v)$ ， $e_2 = (v, u)$ 也算重边而在有向图中不算。**简单图**指没有重边和自环的图，反之被称为**多重图**，若对任意 $u \neq v$ ， $(u, v) \in E$ ，则简单图 G 称为**完全图**。

需要注意，如果题目中没有注明是简单图，要考虑重边和自环带来的影响！

- 度数

在无向图中， $u \in V$ 的**度数** $\deg(u)$ 表示与 u 关联的边数，特别地， (u, u) 对 $\deg(u)$ 应计算两次。

在有向图中， $\deg^+(u)$ 表示以 u 为起点的边数， $\deg^-(u)$ 表示以 u 为终点的边数。

- 简单图

自环，即 $e = (u, u)$ ，**重边**，即 $e_1 = e_2 = (u, v)$ 。需要注意在无向图中 $e_1 = (u, v)$ ， $e_2 = (v, u)$ 也算重边而在有向图中不算。**简单图**指没有重边和自环的图，反之被称为**多重图**，若对任意 $u \neq v$ ， $(u, v) \in E$ ，则简单图 G 称为**完全图**。

需要注意，如果题目中没有注明是简单图，要考虑重边和自环带来的影响！

- 度数

在无向图中， $u \in V$ 的**度数** $\deg(u)$ 表示与 u 关联的边数，特别地， (u, u) 对 $\deg(u)$ 应计算两次。

在有向图中， $\deg^+(u)$ 表示以 u 为起点的边数， $\deg^-(u)$ 表示以 u 为终点的边数。

证明：无向图中 $\sum_{u \in V} \deg(u) = 2m$ ，有向图中 $\sum_{(u \in V)} \deg^+(u) = \sum_{u \in V} \deg^-(u) = m$ 。

- 路径与连通

途径 $W = e_1 e_2 \dots e_k$, 需要满足 $e_{i-1} \cap e_i \neq \emptyset$, 若 $e_i = (v_{i-1}, v_i)$, 可简记为 $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ 或 $v_0 \rightsquigarrow v_k$ 。

行迹 是满足 e_i 两两不同的途径。

回路 是满足 $v_0 = v_k$ 的行迹。

路径 是满足 v_i 两两不同的行迹（也被称为**简单路径**）。

环 是满足除了 $v_0 = v_k$ 外 v_i 互不相同的回路。在无向图中, 若 u, v 满足 $u = v$ 或存在路径 $u \rightsquigarrow v$, 则称 u, v 连通。

- 路径与连通

途径 $W = e_1 e_2 \dots e_k$, 需要满足 $e_{i-1} \cap e_i \neq \emptyset$, 若 $e_i = (v_{i-1}, v_i)$, 可简记为 $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ 或 $v_0 \rightsquigarrow v_k$ 。

行迹 是满足 e_i 两两不同的途径。

回路 是满足 $v_0 = v_k$ 的行迹。

路径 是满足 v_i 两两不同的行迹（也被称为**简单路径**）。

环 是满足除了 $v_0 = v_k$ 外 v_i 互不相同的回路。在无向图中, 若 u, v 满足 $u = v$ 或存在路径 $u \rightsquigarrow v$, 则称 u, v 连通。

证明: 连通关系满足**自反性**, **对称性**, **传递性**。

于是可以将 G 划分为 $G_1, G_2, \dots, G_k$ 共 k 个连通片, 其中两个不同的连通片任意两点不连通。若无特殊说明, 一般认为 G 可以不连通。

- 子图

若 $H = (V', E')$ 有 $V' \subseteq V$ 且 $E' \subseteq E$, 那么 $H \subseteq G$, H 是 G 的**子图**。

若 $E' = \{uv | u, v \in V'\}$, 那么称 H 是 G 的**点导出子图** $G[V']$ 。

若 $V' = \{v | v \in V, \exists uv \in E'\}$, 那么称 H 是 G 的**边导出子图** $G[E']$ 。

定义 $G - v = (V - v, E - \{uv | u \in V, uv \in E\})$, $G - e = (V, E - e)$ 。

• 子图

若 $H = (V', E')$ 有 $V' \subseteq V$ 且 $E' \subseteq E$, 那么 $H \subseteq G$, H 是 G 的**子图**。

若 $E' = \{uv | u, v \in V'\}$, 那么称 H 是 G 的**点导出子图** $G[V']$ 。

若 $V' = \{v | v \in V, \exists uv \in E'\}$, 那么称 H 是 G 的**边导出子图** $G[E']$ 。

定义 $G - v = (V - v, E - \{uv | u \in V, uv \in E\})$, $G - e = (V, E - e)$ 。

• 稀疏图与稠密图

若一张图的边数远小于其点数的平方, 那么它是一张稀疏图。若一张图的边数接近其点数的平方, 那么它是一张稠密图。

这两个概念并没有严格的定义, 一般用于讨论时间复杂度为 $O(|V|^2)$ 的算法与 $O(|E|)$ 的算法的效率差异 (在稠密图上这两种算法效率相当, 而在稀疏图上 $O(|E|)$ 的算法效率明显更高)。

目录

1. 基本概念	2
2. 图基础	7
2.1 图的遍历	8
2.2 二分图	9
2.3 有向无环图	12
2.4 习题	15
3. 最短路	16
3.1 可视化	17
3.2 Dijkstra	18
3.3 Hack Dijkstra	21
3.4 Bellman-Ford	23
3.5 Floyd	28
3.6 Johnson	29
3.7 习题	30

2.1 图的遍历

2.1 图的遍历

- 存图

邻接表：以链表形式在 adj_u 中存取所有与 u 关联的边 e 。一般用 `std::vector` 或链式前向星。

邻接矩阵： $g_{u,v}$ 表示是否存在边 (u,v) 或是 (u,v) 的边权。一般用二维数组。

2.1 图的遍历

- 存图

邻接表：以链表形式在 adj_u 中存取所有与 u 关联的边 e 。一般用 `std::vector` 或链式前向星。

邻接矩阵： $g_{u,v}$ 表示是否存在边 (u,v) 或是 (u,v) 的边权。一般用二维数组。

- 遍历

深度优先搜索 DFS：遍历 u 时将与自己关联的点放在待遍历序列的**最前方**。一般使用递归（本质是栈）。

广度优先搜索 BFS：遍历 u 时将与自己关联的点放在待遍历序列的**最后方**。一般使用队列。

2.2 二分图

2.2 二分图

- P1330 封锁阳光大学

给定一张 n 个点 m 条边的无向图，现在要选出一些点 $w_1, \dots, w_k$ ，使得对于每条边 $e = (u, v)$ ， u, v 都恰好有一个点被选出。

求最少需要选出多少点，若无解输出 Impossible。

$n \leq 10^4$ ， $m \leq 10^5$ 。

2.2 二分图

- P1330 封锁阳光大学

给定一张 n 个点 m 条边的无向图，现在要选出一些点 $w_1, \dots, w_k$ ，使得对于每条边 $e = (u, v)$ ， u, v 都恰好有一个点被选出。

求最少需要选出多少点，若无解输出 Impossible。

$n \leq 10^4$ ， $m \leq 10^5$ 。

可以黑白染色的无向图 G 是二分图。

这里黑白染色合法当且仅当每条边关联的两个点颜色不同。

2.2 二分图

2.2 二分图

二分图是十分简化的，只需要将染色的黑点和白点分开，每条边连接一个黑点一个白点，恰好满足前面题目的限制。

也就是说对于每个连通块，我们选取黑点和白点中较少的一个即可。

何时无解？如何染色？

直接 DFS 即可，如果发现当前颜色与之前矛盾则不是二分图，不合法。

2.2 二分图

二分图是十分简化的，只需要将染色的黑点和白点分开，每条边连接一个黑点一个白点，恰好满足前面题目的限制。

也就是说对于每个连通块，我们选取黑点和白点中较少的一个即可。

何时无解？如何染色？

直接 DFS 即可，如果发现当前颜色与之前矛盾则不是二分图，不合法。

证明： G 是二分图当且仅当 G 中没有奇环（长度为奇数的环）。

2.2 二分图

2.2 二分图

可以通过**扩展域并查集**维护不同连通块的染色结构，便于合并。

2.2 二分图

可以通过**扩展域并查集**维护不同连通块的染色结构，便于合并。

- **CF600F Edge coloring of bipartite graph**

给定一个 $a + b$ 个点 m 条边的无向二分图。将它的每条边染色，使得任意两条相邻（有公共顶点）的边颜色不同。构造一种染色方案，使得用到的颜色数量最少。

$1 \leq a, b \leq 1000, 0 \leq m \leq 10^5$ 。

2.2 二分图

可以通过**扩展域并查集**维护不同连通块的染色结构，便于合并。

- **CF600F Edge coloring of bipartite graph**

给定一个 $a + b$ 个点 m 条边的无向二分图。将它的每条边染色，使得任意两条相邻（有公共顶点）的边颜色不同。构造一种染色方案，使得用到的颜色数量最少。

$1 \leq a, b \leq 1000, 0 \leq m \leq 10^5$ 。

二分图最大匹配、二分图最大权匹配。

这部分我们后面网络流再讲。

2.3 有向无环图

2.3 有向无环图

- B3644 【模板】拓扑排序 / 家谱树

有个人的家族很大，共 n 个人，辈分关系 (u, v) 很杂乱，但保证辈分关系正常，请你整理一下这种关系。给出每个人的后代的信息。输出一个序列，使得每个人的后辈都比那个人后列出。

$1 \leq n, m \leq 5 \times 10^5$ 。

2.3 有向无环图

2.3 有向无环图

我们发现对于点 u ，我们只需要检查有直接连边 (w, u) 的每个 w （后续称为直接祖先）是否被访问即可。

进一步的，我们只需对每个节点维护它的直接祖先还有多少未被访问即可。

当它的所有直接祖先均被访问，那么这个点可以被访问，将其加入待访问队列即可。

复杂度 $O(n)$ ，该算法被称为**拓扑排序**。

2.3 有向无环图

我们发现对于点 u ，我们只需要检查有直接连边 (w, u) 的每个 w （后续称为直接祖先）是否被访问即可。

进一步的，我们只需对每个节点维护它的直接祖先还有多少未被访问即可。

当它的所有直接祖先均被访问，那么这个点可以被访问，将其加入待访问队列即可。

复杂度 $O(n)$ ，该算法被称为**拓扑排序**。

有向无环图，一般简写为 DAG。

证明：拓扑排序在有向无环图的正确性。

DAG 有许多很好的性质，在拓扑排序后相当于找到了一个易于转移的顺序。

2.3 有向无环图

2.3 有向无环图

- P1807 最长路

计算从 1 到 n 的最长路, $1 \leq n \leq 1500$, $0 \leq m \leq 5 \times 10^4$, w 可能为负数。

2.3 有向无环图

- **P1807 最长路**

计算从 1 到 n 的最长路, $1 \leq n \leq 1500$, $0 \leq m \leq 5 \times 10^4$, w 可能为负数。

- **HDU 5638 Toposort**

给定一张 n 个点 m 条边的有向无环图, 找到字典序最小的拓扑序。**特别地**, 你可以至多不满足 k 条边的先后顺序限制。

$1 \leq n \leq 10^5$, $0 \leq m, k \leq 2 \times 10^5$ 。

2.4 习题

- P1525 [NOIP2010 提高组] 关押罪犯
- CF19E Fairy
- P6185 [NOI Online 1 提高组] 序列

目录

1. 基本概念	2
2. 图基础	7
2.1 图的遍历	8
2.2 二分图	9
2.3 有向无环图	12
2.4 习题	15
3. 最短路	16
3.1 可视化	17
3.2 Dijkstra	18
3.3 Hack Dijkstra	21
3.4 Bellman-Ford	23
3.5 Floyd	28
3.6 Johnson	29
3.7 习题	30

3.1 可视化

算法可视化——最短路

<https://visualgo.net/zh/sssp>

3.2 Dijkstra

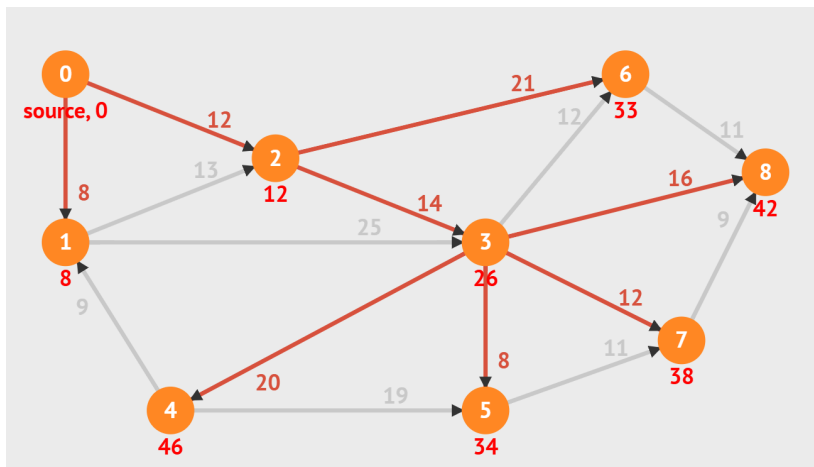
3.2 Dijkstra

- P4779 【模板】单源最短路径（标准版）

给定一个 n 个点， m 条有向边的带非负权图，请你计算从 s 出发到每个点的距离。

数据保证 s 可以抵达任意点。

$1 \leq n \leq 10^5$, $1 \leq m \leq 2 \times 10^5$, $0 \leq w, \sum w \leq 10^9$ 。



3.2 Dijkstra

3.2 Dijkstra

记 dis_u 表示 s 到 u 的最短路径长度。

3.2 Dijkstra

记 dis_u 表示 s 到 u 的最短路径长度。

1. 若 $(u, v, w) \in E$, 有 $\text{dis}_v \leq \text{dis}_u + w$ 。

考虑反证, 若不满足则存在 $s \rightsquigarrow u \rightarrow v$ 长度为 $\text{dis}_u + w < \text{dis}_v$ 。

3.2 Dijkstra

记 dis_u 表示 s 到 u 的最短路径长度。

1. 若 $(u, v, w) \in E$, 有 $\text{dis}_v \leq \text{dis}_u + w$ 。

考虑反证, 若不满足则存在 $s \rightsquigarrow u \rightarrow v$ 长度为 $\text{dis}_u + w < \text{dis}_v$ 。

2. 所有满足 $\text{dis}_v = \text{dis}_u + w$ 的边都在某条 s 到 v 的最短路径上。

3.2 Dijkstra

记 dis_u 表示 s 到 u 的最短路径长度。

1. 若 $(u, v, w) \in E$, 有 $\text{dis}_v \leq \text{dis}_u + w$ 。

考虑反证, 若不满足则存在 $s \rightsquigarrow u \rightarrow v$ 长度为 $\text{dis}_u + w < \text{dis}_v$ 。

2. 所有满足 $\text{dis}_v = \text{dis}_u + w$ 的边都在某条 s 到 v 的最短路径上。

由此我们可以知道存在一个遍历所有点的顺序, 满足 dis_x **单调不降**, 且在遍历到 u 时存在一条 s 到 u 的最短路径使得其上的所有点都在 u 之前被遍历。

Dijkstra 算法正是通过找到这样一个顺序来求出所有点的最短路。

接下来我们描述算法过程, 用 d_u 表示当前时刻**找到的** s 到 u 的最短路径的长度, 显然任意时刻 $d_u \geq \text{dis}_u$, 且我们希望在算法结束时 $d_u = \text{dis}_u$ 。

3.2 Dijkstra

3.2 Dijkstra

初始化显然我们只知道 $d_s = 0$ ，故将其余 d_u 设为 $+\infty$ ，辅助集合 S ，初始为 $\emptyset$ ，辅助集合 T ，初始为全集。

重复执行如下操作直到 T 为空集：

1. 在 T 中找到 d_u 最小的点。
2. 在 T 中删除 u ，在 S 中加入 u ，同时声明 $\text{dis}_u = d_u$ 。
3. 找到所有 $(u, v, w) \in E$ ，若 $d_v > d_u + w$ ，将其改为 $d_u + w$ 。（松弛操作）

3.2 Dijkstra

初始化显然我们只知道 $d_s = 0$ ，故将其余 d_u 设为 $+\infty$ ，辅助集合 S ，初始为 $\emptyset$ ，辅助集合 T ，初始为全集。

重复执行如下操作直到 T 为空集：

1. 在 T 中找到 d_u 最小的点。
2. 在 T 中删除 u ，在 S 中加入 u ，同时声明 $\text{dis}_u = d_u$ 。
3. 找到所有 $(u, v, w) \in E$ ，若 $d_v > d_u + w$ ，将其改为 $d_u + w$ 。 (松弛操作)

通过性质不难说明算法的正确性，留给大家自行证明。

不使用优化的算法复杂度为 $O(n^2 + m)$ ，瓶颈在第一步找到 d_u 最小的点，可以用堆优化将复杂度优化至 $O((n + m) \log(n + m))$ ，进一步可以用斐波那契堆优化至 $O(n \log n + m)$ ，不过一般用不到。

3.3 Hack Dijkstra

3.3 Hack Dijkstra

我们发现 dij 的前置条件是边权非负，那如果边权为负会发生什么？

显然如果出现负环，则算法会死循环，那如果无负环呢？复杂度会变为多少？正确性呢？

3.3 Hack Dijkstra

我们发现 dij 的前置条件是边权非负，那如果边权为负会发生什么？

显然如果出现负环，则算法会死循环，那如果无负环呢？复杂度会变为多少？正确性呢？

于是就诞生了下面这个问题：

构造一张 n 个点 m 条边的带权有向图 G ，使得以 1 为起点的 dij 算法中的 `tot++`；语句至少会执行 10^8 次。

你需要保证 $1 \leq n \leq 70$ ， $1 \leq m \leq 200$ ， $-10^9 \leq w \leq 10^9$ ，图中**无环**。

3.3 Hack Dijkstra

3.3 Hack Dijkstra

数据范围提示我们此时 dij 可以被卡到**指数级**，于是我们观察负权会带来什么。

我们发现若存在一条 (u, v, w) 满足 $w < 0$ ，那么访问完 u 后可能会很快访问 v ，但 u 到 v 可能有更短的路径，比如 $u \rightarrow x \rightarrow v$ ，但 $w(u, x) > 0$ 导致这条路径较晚时才会被访问到。

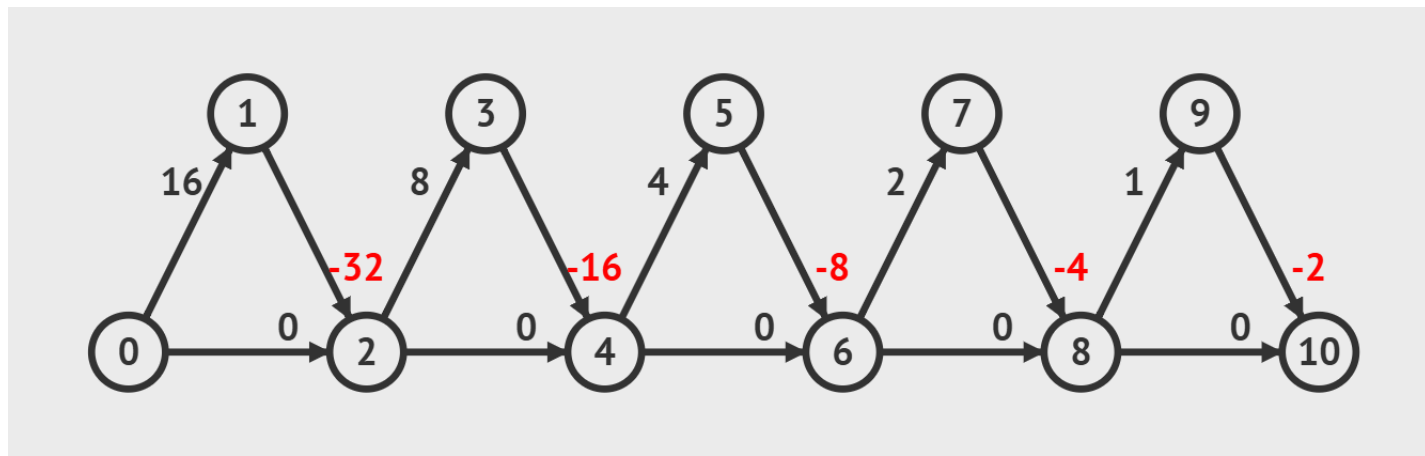
此时非最短的 d_v 就已经有可能把后面的点全都更新了一遍，等由 d_x 更新后又会再走一遍，完成了 $\times 2$ 的复杂度。

3.3 Hack Dijkstra

数据范围提示我们此时 dij 可以被卡到**指数级**，于是我们观察负权会带来什么。

我们发现若存在一条 (u, v, w) 满足 $w < 0$ ，那么访问完 u 后可能会很快访问 v ，但 u 到 v 可能有更短的路径，比如 $u \rightarrow x \rightarrow v$ ，但 $w(u, x) > 0$ 导致这条路径较晚时才会被访问到。

此时非最短的 d_v 就已经有可能把后面的点全都更新了一遍，等由 d_x 更新后又会再走一遍，完成了 $\times 2$ 的复杂度。



3.4 Bellman–Ford

3.4 Bellman–Ford

- **【模板】单源最短路径**

给定一个 n 个点， m 条有向边的带权图，请你计算从 s 出发到每个点的距离。

注意：边权可能为负数。

若 s 能到达一个负环，则最短路可能为 $-\infty$ 。

- **P3385 【模板】负环**

给定一个 n 个点的有向图，请求出图中是否存在**从顶点 1 出发能到达**的负环。

3.4 Bellman–Ford

3.4 Bellman–Ford

边权可能为负数导致 dij 算法中的性质不成立，但有一点是很明显的：如果 s 不能抵达负环，那么任意一条 s 到 u 的最短路径上**至多只有** $n - 1$ 条边。

也就是说如果我们记 $d_{i,u}$ 表示从 s 出发经过 i 条边到达 u 的最短路径长度，那么我们可以松弛 (u, v, w) 这条边，得到一条 s 到 v 经过 $i + 1$ 条边的路径。

然后我们发现 i 取值只能为 $0 \sim n - 1$ ，每轮会把 m 条边均进行一次松弛操作，故复杂度 $O(nm)$ 。

3.4 Bellman-Ford

边权可能为负数导致 dij 算法中的性质不成立，但有一点是很明显的：如果 s 不能抵达负环，那么任意一条 s 到 u 的最短路径上**至多只有** $n - 1$ 条边。

也就是说如果我们记 $d_{i,u}$ 表示从 s 出发经过 i 条边到达 u 的最短路径长度，那么我们可以松弛 (u, v, w) 这条边，得到一条 s 到 v 经过 $i + 1$ 条边的路径。

然后我们发现 i 取值只能为 $0 \sim n - 1$ ，每轮会把 m 条边均进行一次松弛操作，故复杂度 $O(nm)$ 。

但是这样复杂度是卡死的，并且也没有解决负环的问题，考虑优化。

- 经过**严格** i 条边是没什么用的，改为**至多** i 条边。
- 此时可以滚动数组，进一步的，我们发现 $d_{i+1,u}$ 可以直接继承 $d_{i,u}$ 。
- 再进一步，我们发现每轮限制边数也没啥用。

3.4 Bellman–Ford

3.4 Bellman–Ford

然后我们发现无负环的情况 d_x 总会在某一轮之后**保持不变**，此后没有**有效松弛**，而有负环的情况每轮都会有最短路被更新。

于是我们记录每一轮有没有**有效松弛**，超过 n 轮则证明有负环。

3.4 Bellman–Ford

然后我们发现无负环的情况 d_x 总会在某一轮之后**保持不变**，此后没有**有效松弛**，而有负环的情况每轮都会有最短路径被更新。

于是我们记录每一轮有没有**有效松弛**，超过 n 轮则证明有负环。

当然这种写法不是最好的，其实有很多优化方案，比如在 d_v 修改后再松弛其出边，记录最短路径长度而并非轮数来避免最短路径过短导致的溢出等等。

3.4 Bellman–Ford

然后我们发现无负环的情况 d_x 总会在某一轮之后**保持不变**，此后没有**有效松弛**，而有负环的情况每轮都会有最短路被更新。

于是我们记录每一轮有没有**有效松弛**，超过 n 轮则证明有负环。

当然这种写法不是最好的，其实有很多优化方案，比如在 d_u 修改后再松弛其出边，记录最短路径长度而并非轮数来避免最短路径过短导致的溢出等等。

这种队列优化在 OI 界还有一个更为流传的名字：**SPFA**。上述算法是常见优化的 SPFA（其实还有更多优化技巧，不过其他优化方案最劣复杂度可能到达指数级，不推荐使用）。

通常我们可以认为 SPFA 的效率约为 $O(m)$ 。

3.4 Bellman–Ford

3.4 Bellman–Ford

现在就有了一个新的问题：如何将 SPFA 复杂度卡满 $O(nm)$ ？

- 2018 年 7 月 19 日，某位同学在 [NOI Day 1 T1 归程](<https://www.luogu.com.cn/problem/P4768>) 一题里非常熟练地使用了一个广为人知的算法求最短路。
- 然后呢？
- $100 \rightarrow 60$ ；
- $\text{Ag} \rightarrow \text{Cu}$ ；
- 最终，他因此没能与理想的大学达成契约。

3.4 Bellman–Ford

现在就有了一个新的问题：如何将 SPFA 复杂度卡满 $O(nm)$ ？

- 2018 年 7 月 19 日，某位同学在 [NOI Day 1 T1 归程](https://www.luogu.com.cn/problem/P4768) 一题里非常熟练地使用了一个广为人知的算法求最短路。
- 然后呢？
- $100 \rightarrow 60$ ；
- $\text{Ag} \rightarrow \text{Cu}$ ；
- 最终，他因此没能与理想的大学达成契约。

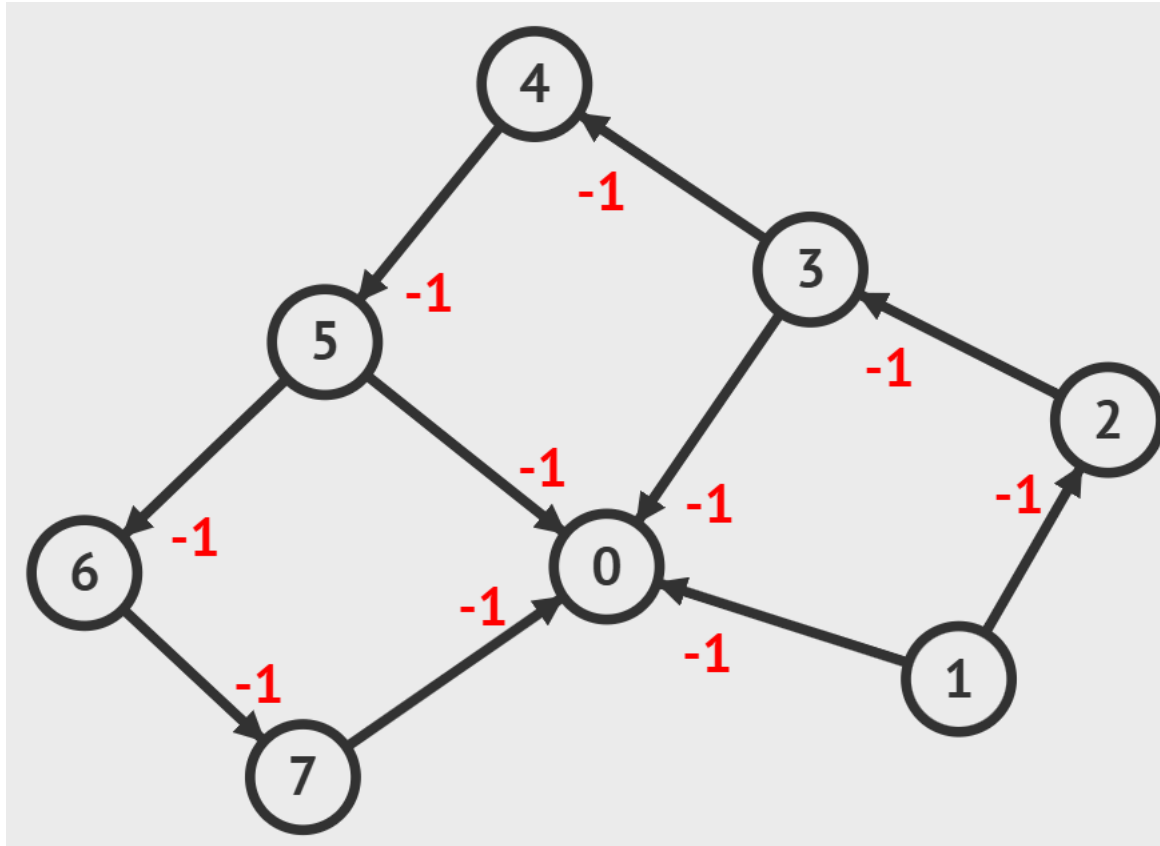
先尝试让一个点的进队次数达到 $O(n)$ 。（注意：在加入**队列唯一**优化后并不容易）

具体操作就是构造边数为 $1, 3, 5, 7, \dots$ 且长度逐渐减小的路径，这样就能使某个点进队 $O(n)$ 次。

然后再在这个点上连接很多出边（菊花）即可做到 $O(n^2)$ 的复杂度，在这张图中 n, m 同阶，也就是 $O(nm)$ 。

3.4 Bellman–Ford

3.4 Bellman–Ford



3.5 Floyd

3.5 Floyd

- B3647 【模板】Floyd

给出一张由 n 个点 m 条边组成的无向连通图。

求出所有点对 (i, j) 之间的最短路径。

$1 \leq n \leq 100, 0 \leq m \leq 4500, w$ 为正整数。

3.5 Floyd

- B3647 【模板】Floyd

给出一张由 n 个点 m 条边组成的无向连通图。

求出所有点对 (i, j) 之间的最短路径。

$1 \leq n \leq 100, 0 \leq m \leq 4500$, w 为正整数。

考虑一个 dp, 记 $f_{k,i,j}$ 表示**路径上**只经过 $\leq k$ 的点得到的从 i 到 j 的最短路径 (当然 $i, j > k$ 是允许的)。

3.5 Floyd

- B3647 【模板】Floyd

给出一张由 n 个点 m 条边组成的无向连通图。

求出所有点对 (i, j) 之间的最短路径。

$1 \leq n \leq 100, 0 \leq m \leq 4500$, w 为正整数。

考虑一个 dp, 记 $f_{k,i,j}$ 表示**路径上**只经过 $\leq k$ 的点得到的从 i 到 j 的最短路径 (当然 $i, j > k$ 是允许的)。

那我们转移就很简单了, 直接判断 k 是否在最短路径上即可。

$$f_{k,i,j} = \min\{f_{k-1,i,j}, f_{k-1,i,k} + f_{k-1,k,j}\}$$

然后我们可以发现 k 这一维可以滚动掉, 复杂度 $O(n^3)$ 。

3.6 Johnson

3.6 Johnson

- P5905 【模板】全源最短路 (Johnson)

给定一个包含 n 个结点和 m 条带权边的有向图，求所有点对间的最短路径长度，一条路径的长度定义为这条路径上所有边的权值和。

$1 \leq n \leq 3 \times 10^3, 1 \leq m \leq 6 \times 10^3$ ，边权可能为负。

3.6 Johnson

- P5905 【模板】全源最短路 (Johnson)

给定一个包含 n 个结点和 m 条带权边的有向图，求所有点对间的最短路径长度，一条路径的长度定义为这条路径上所有边的权值和。

$1 \leq n \leq 3 \times 10^3, 1 \leq m \leq 6 \times 10^3$ ，边权可能为负。

这跟前面的题有什么不一样？

- 有负权：Dij ×
- $n \sim 3 \times 10^3$ ：Floyd ×
- 卡了 SPFA：SPFA ×

3.6 Johnson

• P5905 【模板】全源最短路 (Johnson)

给定一个包含 n 个结点和 m 条带权边的有向图，求所有点对间的最短路径长度，一条路径的长度定义为这条路径上所有边的权值和。

$1 \leq n \leq 3 \times 10^3, 1 \leq m \leq 6 \times 10^3$ ，边权可能为负。

这跟前面的题有什么不一样？

- 有负权：Dij ×
- $n \sim 3 \times 10^3$ ：Floyd ×
- 卡了 SPFA：SPFA ×

这时候我们就需要使用一个超级源点 0，先跑一遍 SPFA 得到一个距离 l_i ，然后把 (u, v, w) 改写为 $(u, v, w + l_u - l_v)$ 。由于最短路性质， $l_u + w \geq l_v$ ，所以新图是非负权图，可以跑 n 遍 Dij。

3.7 习题

- **CF938D Buy a Ticket**
- **P1186 玛丽卡**
- **P3953 [NOIP 2017 提高组] 逛公园**
- **CF1163F Indecisive Taxi Fee**
- **P3008 [USACO11JAN] Roads and Planes G**

本题正解不是 SPFA，但可以通过该题了解一些 SPFA 的奇怪优化。

- **P6190 [NOI Online 1 入门组] 魔法**

Thanks!